APCS 程式設計系列 · U11 · 實作高級 · C++ / Python 對照版

動態規劃 重點講義

遞迴 → 記憶化 → 由下而上 · DP 四步驟 · 爬樓梯與方格路徑 · 最大連續和 · 0/1 背包 · LIS 與 LCS

覺得有幫助？幫老師一個忙

- **訂閱** YouTube「莊老數學教室」：youtube.com/@chuanglaomath
- 看到老師的影片，順手**按個讚**
- 把這份免費講義**分享給同學**：wade-chuang.github.io/jenfoo/apcs（不用註冊，直接下載）

看這個單元的講解影片（YouTube）
【APCS C++ × Python】U11 動態規劃｜DP、背包問題、LIS、LCS｜9 分鐘重點講解
手機掃描 QR code，或點這裡直接看 https://youtu.be/JG_3idHcCm4

1. 從費氏數列看 DP：遞迴 → 記憶化 → 由下而上

費氏數列 f(n) = f(n-1) + f(n-2)。直接照定義寫遞迴，答案對，但同一個 f(k) 會被重算非常多次，n = 50 就要跑好幾分鐘。**動態規劃（DP）的核心只有一句話：每個小問題只算一次，算完記下來。**

C++ 寫法一：純遞迴（很慢）

```cpp
#include <bits/stdc++.h>
using namespace std;

long long f(int n) {
    if (n < 2) return n;           // 終止條件
    return f(n - 1) + f(n - 2);    // 同一個 f(k) 被算很多次
}

int main() {
    int n;
    cin >> n;
    cout << f(n) << "\n";
}
```

Python

```python
def f(n):
    if n < 2:
        return n                   # 終止條件
    return f(n - 1) + f(n - 2)     # 同一個 f(k) 被算很多次

n = int(input())
print(f(n))
```

輸入	輸出
20	6765

C++ 寫法二：記憶化（算過就記下來）

```cpp
#include <bits/stdc++.h>
using namespace std;

long long memo[100];

long long f(int n) {
    if (n < 2) return n;
    if (memo[n] != -1) return memo[n];      // 算過就直接拿
    return memo[n] = f(n - 1) + f(n - 2);   // 算完先記下來
}

int main() {
    memset(memo, -1, sizeof(memo));         // -1 代表還沒算過
    int n;
    cin >> n;
    cout << f(n) << "\n";
}
```

Python

```python
n = int(input())
memo = [-1] * (n + 1)          # -1 代表還沒算過

def f(k):
    if k < 2:
        return k
    if memo[k] != -1:          # 算過就直接拿
        return memo[k]
    memo[k] = f(k - 1) + f(k - 2)   # 算完先記下來
    return memo[k]

print(f(n))
```

輸入	輸出
90	2880067194370816120

C++ 寫法三：由下而上（從小算到大，最常用）

```cpp
int n = 90;
vector<long long> dp(n + 1);
dp[0] = 0; dp[1] = 1;               // 初始值
for (int i = 2; i <= n; i++)        // 由小算到大
    dp[i] = dp[i - 1] + dp[i - 2];  // 轉移式
cout << dp[n] << "\n";
```

Python

```python
n = 90
dp = [0] * (n + 1)
dp[1] = 1                           # 初始值
for i in range(2, n + 1):           # 由小算到大
    dp[i] = dp[i - 1] + dp[i - 2]   # 轉移式
print(dp[n])
```

輸入	輸出
（不需要輸入）	2880067194370816120

寫法	f(n) 的時間	說明
純遞迴	約 $O(1.6^n)$	同一個 f(k) 重算無數次；n = 50 就跑不完
記憶化（由上而下）	O(n)	照遞迴的想法寫，加一個 memo 陣列；但遞迴層數 = n
由下而上（填表）	O(n)	用迴圈從小填到大，沒有遞迴深度問題，考場首選

陷阱提醒 遞迴太深會當掉

【錯】Python 預設遞迴最多約 1000 層（RecursionError）；C++ 幾十萬層也會用完堆疊。

【對】最穩的做法：改寫成由下而上的迴圈。C++ 的 memset 只適合設 0 或 -1。

2. DP 四步驟：以爬樓梯為例

寫 DP 之前，先在紙上把這四件事寫清楚

- **狀態**：dp[i] 代表什麼？一定要用一句完整的中文講出來。例：dp[i] = 走到第 i 階的方法數。
- **轉移**：dp[i] 怎麼由比較小的格子算出來？想「最後一步」。例：最後一步走 1 階或 2 階 → dp[i] = dp[i-1] + dp[i-2]。
- **初始值**：最小的格子直接填。例：dp[0] = 1（站在地面不動，也算一種）。
- **計算順序與答案位置**：算 dp[i] 時，它要用的格子必須已經算好；最後答案在 dp[n]（有些題目要取整張表的最大值）。

C++ 爬樓梯：一次走 1 或 2 階，答案取模

```cpp
#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1000000007;

int main() {
    int n;
    cin >> n;
    vector<long long> dp(n + 1, 0);
    dp[0] = 1;                          // 站在地面：1 種
    for (int i = 1; i <= n; i++) {
        dp[i] = dp[i - 1];              // 最後一步走 1 階
        if (i >= 2)                     // 最後一步走 2 階
            dp[i] = (dp[i] + dp[i - 2]) % MOD;
    }
    cout << dp[n] << "\n";
}
```

Python

```python
n = int(input())
MOD = 1000000007
dp = [0] * (n + 1)
dp[0] = 1                       # 站在地面：1 種
for i in range(1, n + 1):
    dp[i] = dp[i - 1]           # 最後一步走 1 階
    if i >= 2:                  # 最後一步走 2 階
        dp[i] = (dp[i] + dp[i - 2]) % MOD
print(dp[n])
```

輸入	輸出
100	782204094

陷阱提醒 爬樓梯三大錯

【錯】初始值寫錯：dp[0] 設成 0，後面全部變 0；dp[1] 和 dp[2] 自己手算對一下。

【錯】忘記取模：C++ 會溢位、Python 數字會變成幾萬位數。每一次相加後都要 % MOD。

【錯】C++ 兩個小於 MOD 的數相乘會到 10^{18}，一定要用 long long。

3. 方格路徑（含障礙）

R × C 的方格，只能往右或往下走，「#」是障礙不能進入。**狀態**：dp[i][j] = 從左上角走到 (i, j) 的方法數。**轉移**：最後一步不是從上面來，就是從左邊來，所以 dp[i][j] = dp[i-1][j] + dp[i][j-1]；障礙格直接是 0。

C++ 方格路徑：障礙格是 0，其他格 = 上 + 左

```cpp
#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1000000007;

int main() {
    int r, c;
    cin >> r >> c;
    vector<string> g(r);
    for (auto &s : g) cin >> s;
    vector<vector<long long>> dp(r, vector<long long>(c, 0));
    for (int i = 0; i < r; i++)
        for (int j = 0; j < c; j++) {
            if (g[i][j] == '#') continue;            // 障礙：0 種
            if (i == 0 && j == 0) { dp[i][j] = 1; continue; }
            long long up = (i > 0) ? dp[i - 1][j] : 0;
            long long left = (j > 0) ? dp[i][j - 1] : 0;
            dp[i][j] = (up + left) % MOD;
        }
    cout << dp[r - 1][c - 1] << "\n";
}
```

Python

```python
import sys
data = sys.stdin.read().split()
r, c = int(data[0]), int(data[1])
g = data[2:2 + r]
MOD = 1000000007
dp = [[0] * c for _ in range(r)]
for i in range(r):
    for j in range(c):
        if g[i][j] == "#":
            continue                    # 障礙：0 種
        if i == 0 and j == 0:
            dp[i][j] = 1
            continue
        up = dp[i - 1][j] if i > 0 else 0
        left = dp[i][j - 1] if j > 0 else 0
        dp[i][j] = (up + left) % MOD
print(dp[r - 1][c - 1])
```

輸入	輸出
3 4 .... .#..	4

陷阱提醒

【錯】把第一列、第一行全部直接設成 1：如果第一列中間有障礙，障礙「後面」的格子其實走不到，應該是 0。

【對】用「上 + 左，超出邊界當 0」統一處理，第一列、第一行就不用特別寫。

【錯】起點本身是障礙時，答案是 0；程式要先判斷障礙，再設起點為 1。

4. 最大連續子陣列和

狀態：dp[i] =「以 a[i] 結尾」的連續區段中，最大的總和。**轉移**：a[i] 要嘛接在前面那一段後面，要嘛自己重新開始：dp[i] = max(dp[i-1] + a[i], a[i])。**答案**不是 dp[n-1]，而是所有 dp[i] 裡最大的那一個。

C++ 最大連續和：接上去，還是重新開始

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    long long cur = a[0], best = a[0];   // 用第一個數當初始值
    for (int i = 1; i < n; i++) {
        cur = max(cur + a[i], a[i]);     // 接上去，或重新開始
        best = max(best, cur);           // 答案取所有 dp 的最大
    }
    cout << best << "\n";
}
```

Python

```python
n = int(input())
a = list(map(int, input().split()))
cur = best = a[0]            # 用第一個數當初始值
for x in a[1:]:
    cur = max(cur + x, x)    # 接上去，或重新開始
    best = max(best, cur)    # 答案取所有 dp 的最大
print(best)
```

輸入	輸出
9 -2 1 -3 4 -1 2 1 -5 4	6

陷阱提醒

【錯】cur 和 best 從 0 開始：全部都是負數時會錯印 0（題目通常規定至少選一個數）。

【錯】只輸出最後一格 dp[n-1]：最好的區段不一定在最後面結束。

【錯】C++ 的總和可能到 2×10^{14}，要用 long long。

5. 0/1 背包：二維表格，再壓成一維（要倒著跑）

n 個物品，第 i 個重 w_i、價值 v_i，每個物品最多拿一次，背包容量 W，求最大價值。**狀態**：dp[i][c] = 只考慮前 i 個物品、容量 c 時的最大價值。**轉移**：第 i 個不拿 → dp[i-1][c]；拿 → dp[i-1][c - w_i] + v_i，兩者取大。

C++ 0/1 背包：二維表格

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, W;
    cin >> n >> W;
    vector<int> w(n + 1), v(n + 1);
    for (int i = 1; i <= n; i++) cin >> w[i] >> v[i];
    vector<vector<long long>> dp(n + 1,
                                 vector<long long>(W + 1, 0));
    for (int i = 1; i <= n; i++)
        for (int c = 0; c <= W; c++) {
            dp[i][c] = dp[i - 1][c];              // 不拿第 i 個
            if (c >= w[i])                        // 拿第 i 個
                dp[i][c] = max(dp[i][c],
                               dp[i - 1][c - w[i]] + v[i]);
        }
    cout << dp[n][W] << "\n";
}
```

Python

```python
import sys
data = list(map(int, sys.stdin.read().split()))
n, W = data[0], data[1]
w = [0] + data[2::2][:n]
v = [0] + data[3::2][:n]
dp = [[0] * (W + 1) for _ in range(n + 1)]
for i in range(1, n + 1):
    for c in range(W + 1):
        dp[i][c] = dp[i - 1][c]          # 不拿第 i 個
        if c >= w[i]:                    # 拿第 i 個
            take = dp[i - 1][c - w[i]] + v[i]
            dp[i][c] = max(dp[i][c], take)
print(dp[n][W])
```

輸入	輸出
3 6 2 3 3 4 5 6	7

C++ 0/1 背包：一維陣列，容量倒著跑

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, W;
    cin >> n >> W;
    vector<long long> dp(W + 1, 0);    // dp[c]：容量 c 的最大價值
    for (int i = 0; i < n; i++) {
        int w, v;
        cin >> w >> v;
        for (int c = W; c >= w; c--)    // 倒著跑！
            dp[c] = max(dp[c], dp[c - w] + v);
    }
    cout << dp[W] << "\n";
}
```

Python

```python
import sys
data = list(map(int, sys.stdin.read().split()))
n, W = data[0], data[1]
dp = [0] * (W + 1)              # dp[c]：容量 c 的最大價值
for i in range(n):
    w, v = data[2 + 2 * i], data[3 + 2 * i]
    for c in range(W, w - 1, -1):   # 倒著跑！
        dp[c] = max(dp[c], dp[c - w] + v)
print(dp[W])
```

輸入	輸出
3 6 2 3 3 4 5 6	7

C++ 正著跑會怎樣？同一個物品被拿了好幾次

```cpp
int W = 6, w[3] = {2, 3, 5}, v[3] = {3, 4, 6};
long long a[7] = {0}, b[7] = {0};
for (int i = 0; i < 3; i++) {
    for (int c = W; c >= w[i]; c--)     // 倒著跑：0/1 背包
        a[c] = max(a[c], a[c - w[i]] + v[i]);
    for (int c = w[i]; c <= W; c++)     // 正著跑：無限背包
        b[c] = max(b[c], b[c - w[i]] + v[i]);
}
cout << a[W] << " " << b[W] << "\n";
```

Python

```python
W, w, v = 6, [2, 3, 5], [3, 4, 6]
a = [0] * (W + 1)
b = [0] * (W + 1)
for i in range(3):
    for c in range(W, w[i] - 1, -1):   # 倒著跑：0/1 背包
        a[c] = max(a[c], a[c - w[i]] + v[i])
    for c in range(w[i], W + 1):       # 正著跑：無限背包
        b[c] = max(b[c], b[c - w[i]] + v[i])
print(a[W], b[W])
```

輸入	輸出
（不需要輸入）	7 9

正著跑時，dp[c - w] 可能「這一輪」已經放過同一個物品，等於重量 2 的物品被拿了 3 次（3 + 3 + 3 = 9）。

陷阱提醒 背包題必查

【錯】一維 0/1 背包的容量迴圈正著跑 → 變成「每個物品可以拿無限次」的無限背包，答案偏大。

【對】倒著跑時，dp[c - w] 還是「上一個物品」的舊值，保證每個物品最多拿一次。

- 時間 O(nW)、空間 O(W)。n × W 超過約 10^7 ~ 10^8 就要小心超時（Python 更早）。

6. 最長遞增子序列（LIS）$O(n^2)$

子序列不必連續，但順序不能變。**狀態**：dp[i] = 以 a[i] 結尾的最長嚴格遞增子序列長度。**轉移**：往前找所有 j < i 且 a[j] < a[i]，接在它後面：dp[i] = max(dp[j]) + 1；找不到就是 1。**答案**是所有 dp[i] 的最大值。

C++ LIS：每個數往前找比它小的接上去

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<int> a(n), dp(n, 1);        // 每個數自己就是長度 1
    for (auto &x : a) cin >> x;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < i; j++)
            if (a[j] < a[i])           // 嚴格遞增用 <
                dp[i] = max(dp[i], dp[j] + 1);
    cout << *max_element(dp.begin(), dp.end()) << "\n";
}
```

Python

```python
n = int(input())
a = list(map(int, input().split()))
dp = [1] * n                   # 每個數自己就是長度 1
for i in range(n):
    for j in range(i):
        if a[j] < a[i]:        # 嚴格遞增用 <
            dp[i] = max(dp[i], dp[j] + 1)
print(max(dp))                 # 答案取整張表的最大
```

輸入	輸出
8 3 1 4 1 5 9 2 6	4

預告：O(n log n) 的 LIS

- $O(n^2)$ 在 n ≦ 5000 左右可以用（Python 建議 n ≦ 2000）。
- n 到 2×10^5 時，要改用「tails 陣列 + 二分搜尋」：tails[k] 記錄長度 k + 1 的遞增子序列，結尾最小可以是多少。
- 每讀一個數，用二分搜尋（C++ lower_bound、Python bisect_left）找位置替換，時間 O(n log n)。

7. 最長共同子序列（LCS）與經典 DP 總整理

狀態：dp[i][j] = 字串 a 的前 i 個字元、b 的前 j 個字元，最長共同子序列的長度。**轉移**：a 的第 i 個字元等於 b 的第 j 個字元 → 左上角 + 1；不相等 → 上面和左邊取大。**初始值**：第 0 列、第 0 行（空字串）全部是 0。

C++ LCS：相同取左上 + 1，不同取上、左較大

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    string a, b;
    cin >> a >> b;
    int n = a.size(), m = b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++) {
            if (a[i - 1] == b[j - 1])        // 字串從 0 開始編號
                dp[i][j] = dp[i - 1][j - 1] + 1;
            else
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
        }
    cout << dp[n][m] << "\n";
}
```

Python

```python
a = input().strip()
b = input().strip()
n, m = len(a), len(b)
dp = [[0] * (m + 1) for _ in range(n + 1)]
for i in range(1, n + 1):
    for j in range(1, m + 1):
        if a[i - 1] == b[j - 1]:     # 字串從 0 開始編號
            dp[i][j] = dp[i - 1][j - 1] + 1
        else:
            dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
print(dp[n][m])
```

輸入	輸出
ABCBDAB BDCABA	4

題型	dp 代表什麼	轉移	時間
爬樓梯	dp[i]：走到第 i 階的方法數	dp[i-1] + dp[i-2]	O(n)
方格路徑	dp[i][j]：走到 (i, j) 的方法數	上 + 左；障礙格為 0	O(RC)
最大連續和	dp[i]：以 a[i] 結尾的最大和	max(dp[i-1] + a[i], a[i])	O(n)
0/1 背包	dp[c]：容量 c 的最大價值	max(dp[c], dp[c-w] + v)，c 倒著跑	O(nW)
LIS	dp[i]：以 a[i] 結尾的最長長度	j < i 且 a[j] < a[i] 的 dp[j] 最大值 + 1	$O(n^2)$
LCS	dp[i][j]：a 前 i 個、b 前 j 個	相同：左上 + 1；不同：max(上, 左)	O(nm)

讀完這份，你應該要會

- □ 看到遞迴重複計算，會改成記憶化，再改成由下而上的迴圈
- □ 寫 DP 前會先寫出：狀態、轉移、初始值、計算順序與答案位置
- □ 方法數的題目每一步都會 % 1000000007，初始值 dp[0] 會自己手算確認
- □ 最大連續和知道答案是「所有 dp 的最大值」，全負數也不會錯
- □ 0/1 背包會寫二維與一維，知道一維為什麼要倒著跑
- □ LIS、LCS 的 dp 表可以在紙上一格一格填出來

有一項沒把握，就回去重看那一段，再做練習題。

看影片講解、問問題

YouTube 莊老數學教室　　IG @jf_chuang　　LINE 諮詢 jenfoo2021

覺得有幫助？幫老師一個忙

- **訂閱** YouTube「莊老數學教室」：youtube.com/@chuanglaomath
- 看到老師的影片，順手**按個讚**
- 把這份免費講義**分享給同學**：wade-chuang.github.io/jenfoo/apcs（不用註冊，直接下載）